

Final Report

Ryan Waterman

Table of Contents

Executive Summary3

Approach and Data4

Detailed Findings and Evaluation6

Business Recommendations15

Appendices17

Executive Summary

The intent of the performed analysis was to identify a dense, feed-forward neural network that could predict the probability of customers cancelling their bookings with ABC Hotels based on a data set containing over 35,000 bookings. Based on this goal, four different model architectures were developed and compared using ROC Curves, AUC, Validation Accuracy, Recall, Precision, F1-Score, Training Time, and Inference Time.

After a thorough data exploration and preliminary analysis phase, a final round of model training and evaluation was completed. The outcome of this evaluation required a thoughtful interpretation of the outcome, considering the nuances of the comparison metrics in the scope of this application. A summary of these results can be seen below:

Model Type	Validation Accuracy	Recall	Precision	F1-Score	Training Time (s)	Inference Time (s)
1	97.27%	91.6%	100.00%	0.96	0.0876	0.7168
2	99.97%	99.91%	100.00%	1	1.4565	1.8502
3	100.00%	100.00%	100.00%	1	0.8250	0.9488
4	100.00%	100.00%	100.00%	1	0.6358	0.8339

Table 1: Model Result Summary

The model's architectures can be found in the *Detailed Findings and Evaluation* section, as well as an in-depth review of the results. Ultimately, Model 3 is the optimal model for predicting cancellation probability. Model 4 outperforms Model 3, but Model 4 is a dense neural network comprised of only linear layers. The lack of activation functions in Model 4 is a critical issue; although it performs well on validation data in the current dataset, there could be higher level features that it did not learn that could make Model 3 superior on novel data. A minimal training and inference time trade-off is worth the security of generalized accuracy provided by Model 3.

A key detail to consider is that the prediction accuracies were determined by rounding the model outputs, which are probabilities between 0 and 1, and then determining the frequency of correct binary predictions against the existing data. While this is required to train the model for classification, the end user is expecting the model output to be a cancellation probability. Each of these models produces this probability as part of their output, and during deployment in a production setting, the probability for either class (cancelled or not cancelled) will be evaluated to determine cancellation risk.

Approach and Data

As suggested in the *Executive Summary* section, the approach taken to train deep neural networks to predict cancellation probability stems from training classification models. In order for the losses to be computed to drive model learning, the models need binary classes to compare the accuracy of the prediction to the ground truth. The business need is to identify the cancellation probability for a given customer based on data about their stay, which is a core component of the classification model output. In production, ABC Hotels will strip off the probability rounding step and analyze the raw probability of either class that the model outputs. Using this approach for the “cancelled” class, ABC Hotels can start marketing toward subsets of the cancellation distribution, such as all customers with a cancellation probability higher than 80 percent.

The data preparation process conducted during the initial experimentation can be broken into three subsections: one-hot encoding and train-test-validation data splitting. Each of these subsections are explained, in detail, below.

One-hot encoding was performed on all categorical variables using the algorithm described below:

1. Determine the categorical and quantitative variables. This was performed using the `describe()` function from pandas, which only outputs columns with numerical data.

```
data = pd.read_csv('project_data.csv')

quantitative = set((data.drop('Booking_ID', axis=1).describe().columns))
categorical = set((data.drop(['Booking_ID', 'arrival_date', 'booking_status'], axis=1).columns)) - quantitative
```

2. Iterate through the categorical variable columns in the dataset, then identify the unique values for each column. For each unique value, generate a new column (named after the unique value), then generate an array with 1 at the indices of the unique value for the initial column and 0 elsewhere.

```
for cat in list(categorical):
    # Find the unique values for the column
    unique_vals = data[cat].unique()

    # For each unique value, create a new column with the value name
    for val in unique_vals:
        data[val] = np.where(data[cat].values==val, 1, 0)
```

3. Note that `booking_status` was omitted from the categorical columns. This is the Y variable that the model will be predicting, therefore it will remain as one column as it is encoded to 1 and 0.

```
encoded_data[encoded_data['booking_status'] == "not_canceled"] = 0
encoded_data[encoded_data['booking_status'] == "canceled"] = 1
```

The train-test-validation split selected for the entirety of the analysis was 70% train, 20% test, and 10% validation with a random seed generated by `torch.seed()`. The algorithm used to split the data can be found below:

```
def _split_data(self):
    n_samples = len(self.X)
    n_test = int(0.20 * n_samples)
    n_val = int(0.10 * n_samples)

    shuffled_indices = torch.randperm(n_samples)

    train_indices = shuffled_indices[:-n_val-n_test]
    test_indices = shuffled_indices[-n_val-n_test:-n_val]
    val_indices = shuffled_indices[-n_val:]

    # Verify the indices are all unique
    train_set = set(train_indices.numpy())
    test_set = set(test_indices.numpy())
    val_set = set(val_indices.numpy())

    if not len(train_set & test_set) and not len(train_set & val_set) and not len(test_set & val_set):
        print("Train, Test, and Validation data is verified as unique.")
    else:
        print("DATA LEAK.")

    #Assign train data
    self.X_train = self.X[train_indices].to('cuda')
    self.Y_train = self.Y[train_indices].to('cuda')

    #Assign train data
    self.X_test = self.X[test_indices].to('cuda')
    self.Y_test = self.Y[test_indices].to('cuda')

    #Assign val data
    self.X_val = self.X[val_indices].to('cuda')
    self.Y_val = self.Y[val_indices].to('cuda')
```

In addition to one-hot encoding and train-test-validation split, there were omitted and feature engineered variables. As discussed in the Analytic Plan, the Booking ID variable has been omitted, as it is a unique identifier for each observation. Additionally, the Arrival Date variable has been modified to be an integer value representing only the arrival month. The intent of this feature is to capture seasonality in the data.

Detailed Findings and Evaluation

Model test was completed across 4 model architectures utilizing a custom “Model Handler” framework. The intent of developing this framework was to dramatically reduce the amount of redundant code, as well as providing a consistent set of performance metrics to evaluate the models against each other. It is important to note that each of the models were trained on the same data, determined by the code block, below:

```
torch.seed()
X = encoded_data.drop(["booking_status"], axis=1)
Y = encoded_data['booking_status'].astype(np.int64)
input_features = len(encoded_data.drop('booking_status', axis=1).columns)
labels = ['Not Canceled', 'Canceled']
```

In this phase of development, early stoppage was incorporated to prevent overfitting. The implementation for early stoppage was based on the slope of both training and test losses across the training iterations. The implementation can be seen below:

```
def train(self, n_epochs, lr = 1e-3, slope_interval = 10, stop_slope = 0.000001):
    self.training_losses = []
    self.test_losses = []
    self.optimizer = self.optimizer(self.model.parameters(), lr)
    start_time = time()

    for epoch in range(1, n_epochs+1):
        p_train = self.model(self.X_train).to('cuda')
        train_loss = self.loss_fn(p_train, self.Y_train)
        self.training_losses.append(train_loss.to('cpu').item())

        p_test = self.model(self.X_test)
        loss_test = self.loss_fn(p_test, self.Y_test)
        self.test_losses.append(loss_test.to('cpu').item())

        self.optimizer.zero_grad()
        train_loss.backward() # <2>
        self.optimizer.step()

        if len(self.training_losses) > slope_interval:
            self.train_loss_slope = ((self.training_losses[-1] - self.training_losses[-slope_interval]) / slope_interval)
            self.test_loss_slope = ((self.test_losses[-1] - self.test_losses[-slope_interval]) / slope_interval)
            print(f"Epoch {epoch}\tTraining loss {train_loss.item():.4f}\tTest loss {loss_test.item():.4f}\t"
                  f"Train Slope: {self.train_loss_slope:.20f}\tTest Slope: {self.test_loss_slope:.20f}""", end="\r")

            if abs(self.train_loss_slope) < stop_slope and abs(self.test_loss_slope) < stop_slope:
                break

    end_time = time()
    print('\n')
    print(f"Total training time: {end_time - start_time} seconds")
    print('\n')
    self.plot_losses(self.training_losses, self.test_losses, 'Training vs. Test Loss')
```

The parameters `slope_interval` and `stop_slope` determine when the early stoppage occurs. This algorithm utilizes a sliding window of the last `slope_interval` test and training losses, then computes the slope for that set of losses. If the slope is less than `stop_slope`, then training is halted. During the initial training stage for each model, `slope_interval` was set to 100 and `stop_slope` was set to `1e-4` in order to keep consistency.

The model architectures, as well as the Learning Curves, Confusion Matrices, ROC Curves, AUC scores, training time, and inference time, are described under each of the “**MODEL**” headers, below.

MODEL 1

```
model_1 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=10)),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear2', nn.Linear(in_features=10, out_features=5)),
    ('hidden_activation', nn.Tanh()),
    ('output_linear', nn.Linear(in_features=5, out_features=2))
]))

model_1 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_1,
    optimizer = optim.SGD,
    loss_fn = nn.CrossEntropyLoss
)

model_1.train(100000, 1e-4, 100, 1e-4)
model_1.eval_model(labels)
```

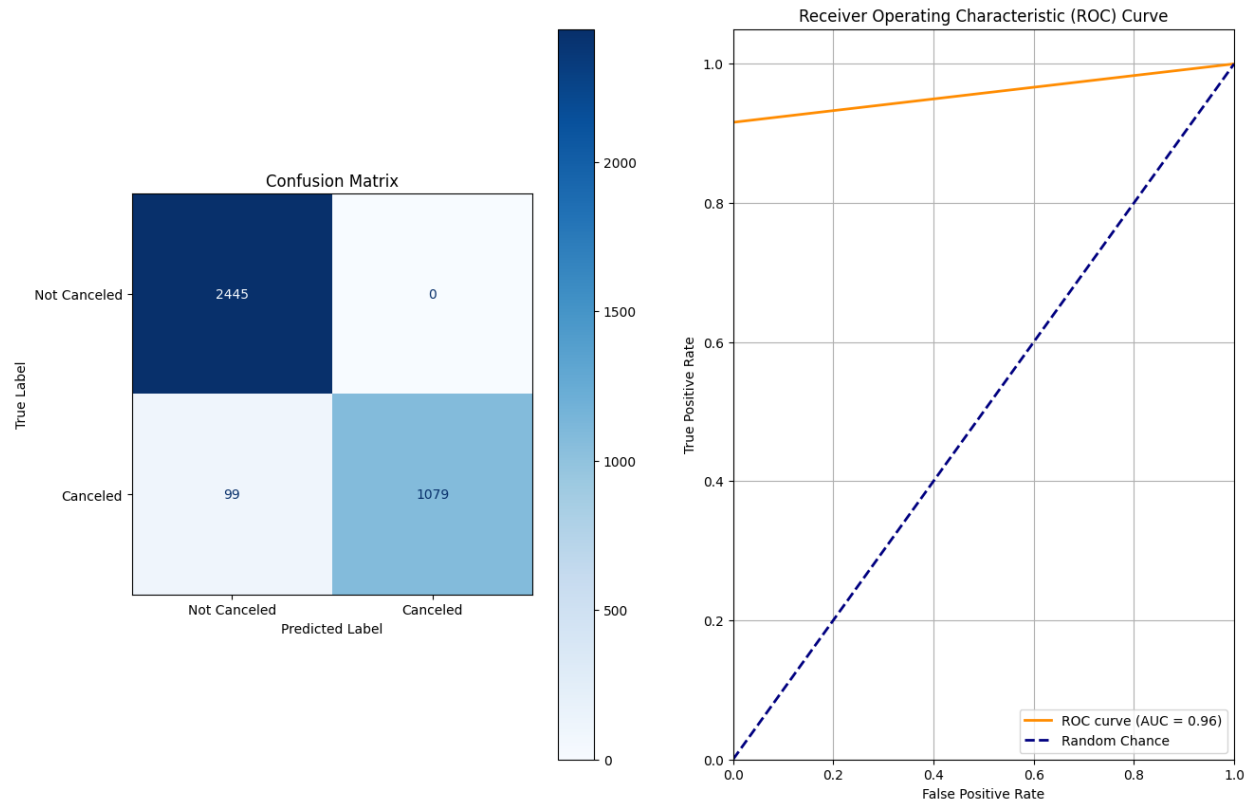


Model 1 Architecture and Loss Plot

Model 1 was trained using a Stochastic Gradient Descent optimizer and Cross Entropy Loss function. Cross Entropy Loss was selected as it outputs a log loss value for each class in a

classification model. In this application, there are two classes, canceled and not canceled, and the log loss of the prediction can be used to interpret the prediction probability of each class. Note that the early stoppage occurred erroneously in this case due to the variance in the losses over the training iterations.

One dropout layer was included with a 20% dropout rate, as well as a Tanh activation function, which assist in adding non-linearity and addressing vanishing gradients. This is an intentionally small model, so these are the only activation/randomization functions used.



Model 1 Validation Performance Metrics

MODEL 2

```
model_2 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=200)),
    ('hidden_linear2', nn.Linear(in_features=200, out_features=500)),
    ('relu1', nn.ReLU()),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear3', nn.Linear(in_features=500, out_features=400)),
    ('relu2', nn.ReLU()),
    ('dropout2', nn.Dropout(0.3)),
    ('hidden_linear4', nn.Linear(in_features=400, out_features=200)),
    ('lrelu1', nn.LeakyReLU()),
    ('dropout3', nn.Dropout(0.3)),
    ('hidden_linear6', nn.Linear(in_features=200, out_features=150)),
    ('relu3', nn.ReLU()),
    ('dropout4', nn.Dropout(0.2)),
    ('hidden_linear7', nn.Linear(in_features=150, out_features=100)),
    ('hidden_linear8', nn.Linear(in_features=100, out_features=50)),
    ('relu4', nn.ReLU()),
    ('dropout5', nn.Dropout(0.3)),
    ('hidden_linear9', nn.Linear(in_features=50, out_features=20)),
    ('lrelu2', nn.LeakyReLU()),
    ('dropout6', nn.Dropout(0.3)),
    ('hidden_linear10', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.Softmax())
]))

model_2 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_2,
    optimizer = optim.AdamW,
    loss_fn = nn.CrossEntropyLoss
)

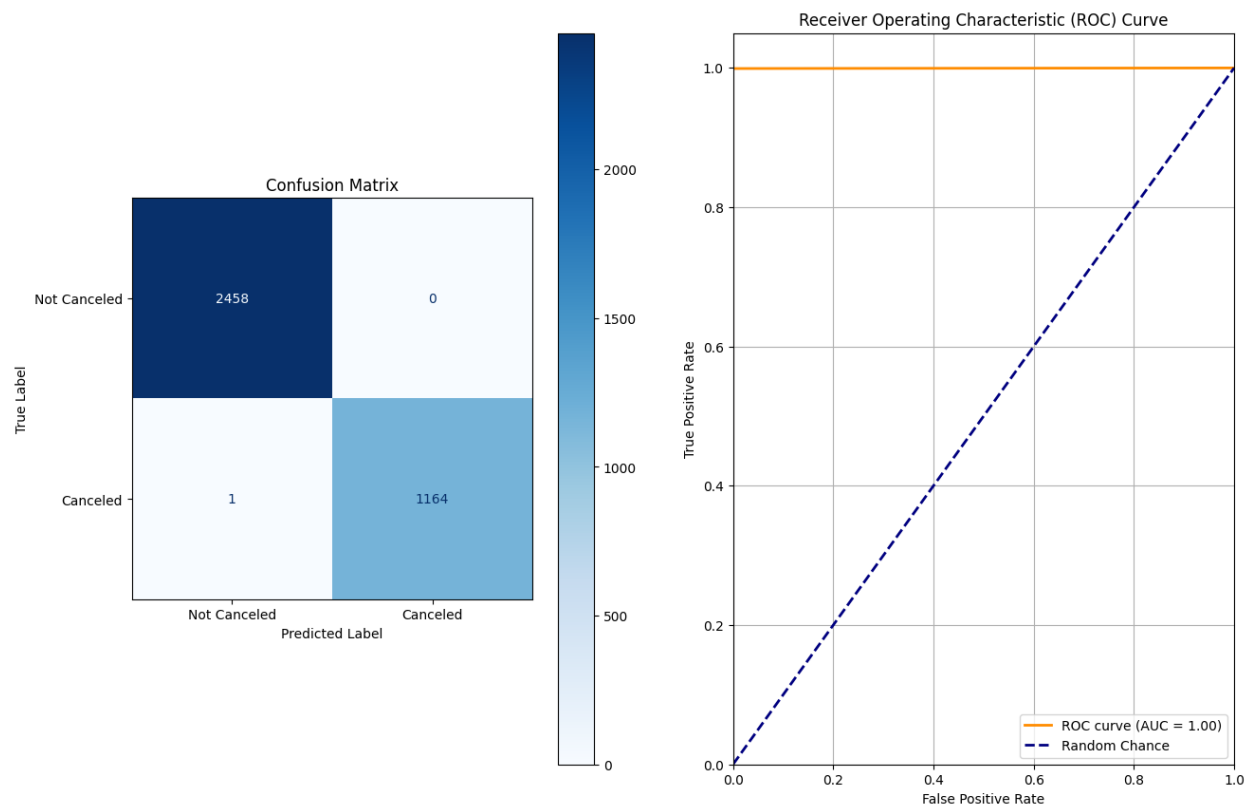
model_2.train(100000, 1e-4, 100, 1e-4)
model_2.eval_model(labels)
```



Model 2 Architecture and Loss Plot

Model 2 was designed to identify the impact of scale on training and model performance. Similar to Model 1, Cross Entropy Loss was selected as the loss function, but the optimizer was changed to AdamW. The AdamW optimizer was selected because the model is larger, and the adaptive learning rate intuitively seems like a better optimization function for deeper models.

This model utilized a Softmax output layer, as well as more consistent activation-dropout layer pairing throughout. Softmax outputs a PDF for each of the classes, which simplifies the process of identifying the cancellation probability when running inference on the deployed model.



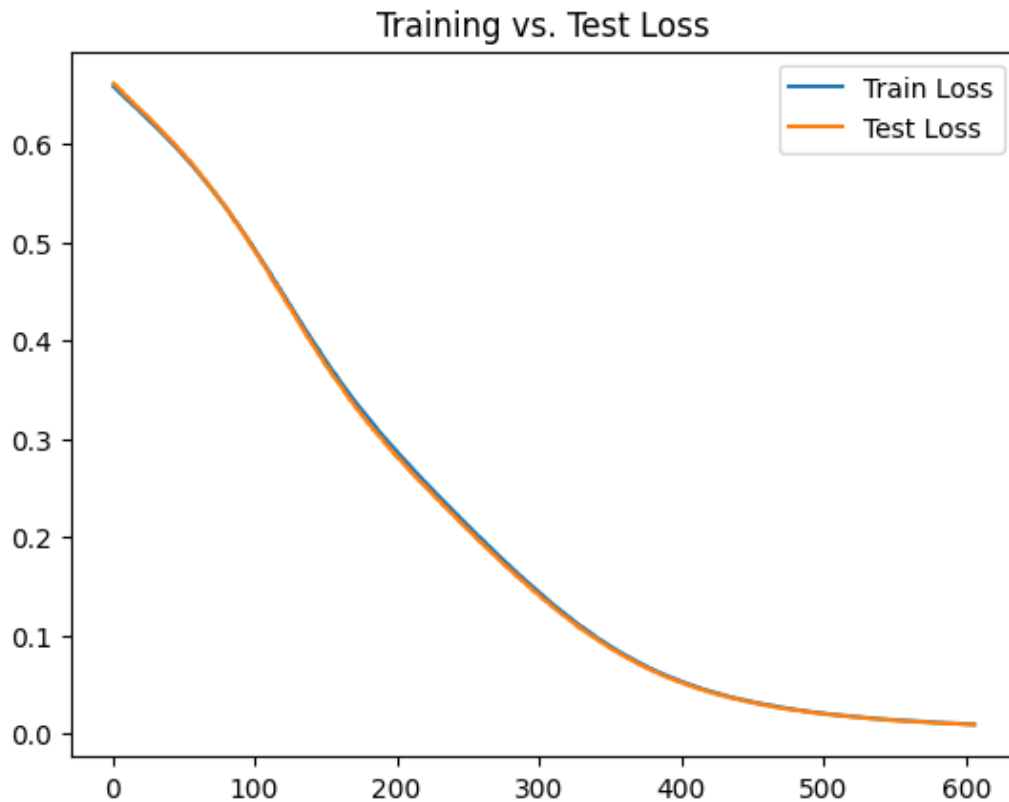
Model 2 Validation Performance Metrics

MODEL 3

```
model_3 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=20)),
    ('hidden_linear2', nn.Linear(in_features=20, out_features=50)),
    ('dropout1', nn.Dropout(0.2)),
    ('hidden_linear3', nn.Linear(in_features=50, out_features=40)),
    ('dropout2', nn.Dropout(0.3)),
    ('hidden_linear4', nn.Linear(in_features=40, out_features=20)),
    ('hidden_activation', nn.LeakyReLU()),
    ('hidden_linear6', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.LogSoftmax())
]))

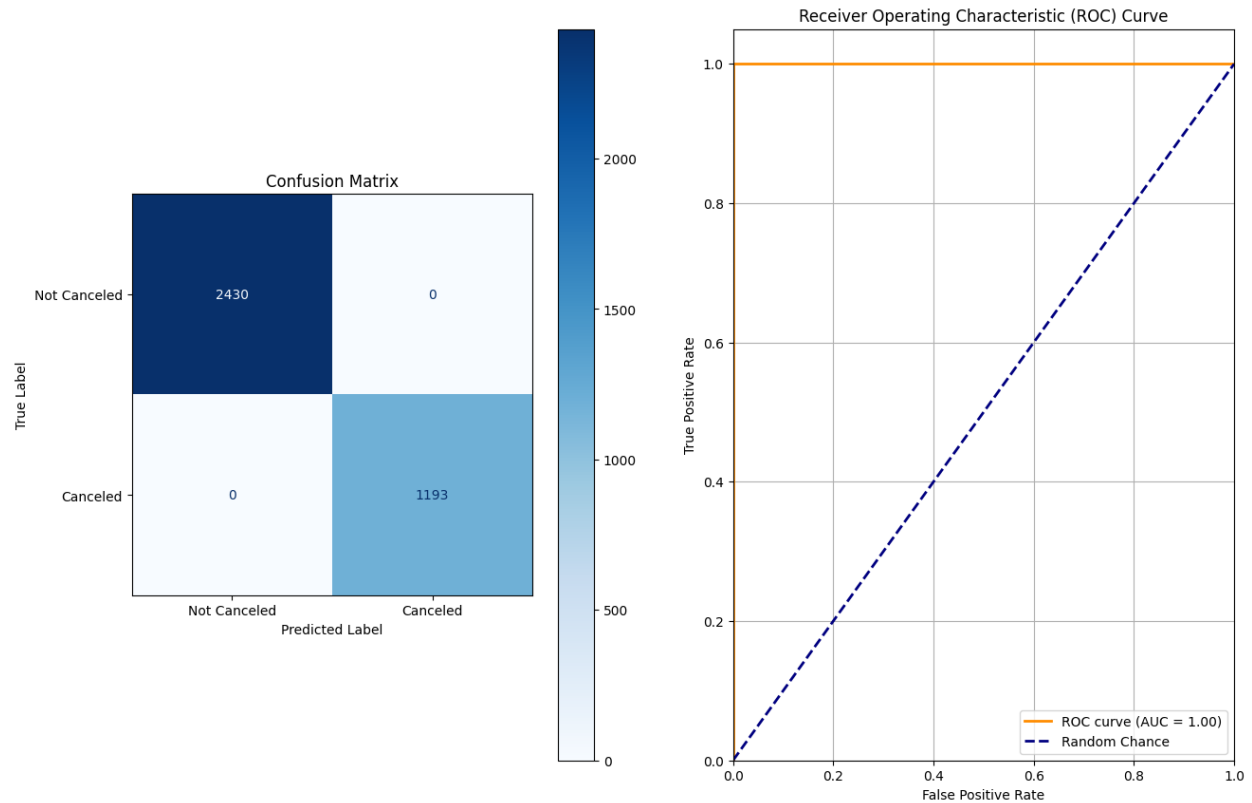
model_3 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_3,
    optimizer = optim.AdamW,
    loss_fn = nn.NLLLoss
)

model_3.train(100000, 1e-4, 100, 1e-4)
model_3.eval_model(labels)
```



Model 3 Architecture and Loss Plot

After identifying a dramatic increase in training time on Model 2, Model 3 aims to achieve similar performance without the training overhead of a much larger model. The output layer was changed to LogSoftmax to accommodate the Negative Log Likelihood Loss function, which is similar to Cross Entropy, but instead maximizes the probability of the correct class.



Model 3 Performance Metrics

MODEL 4

```
model_4 = nn.Sequential(OrderedDict([
    ('hidden_linear1', nn.Linear(in_features=input_features, out_features=10)),
    ('hidden_linear2', nn.Linear(in_features=10, out_features=20)),
    ('hidden_linear3', nn.Linear(in_features=20, out_features=30)),
    ('hidden_linear4', nn.Linear(in_features=30, out_features=20)),
    ('hidden_linear5', nn.Linear(in_features=20, out_features=2)),
    ('output', nn.LogSoftmax())
]))

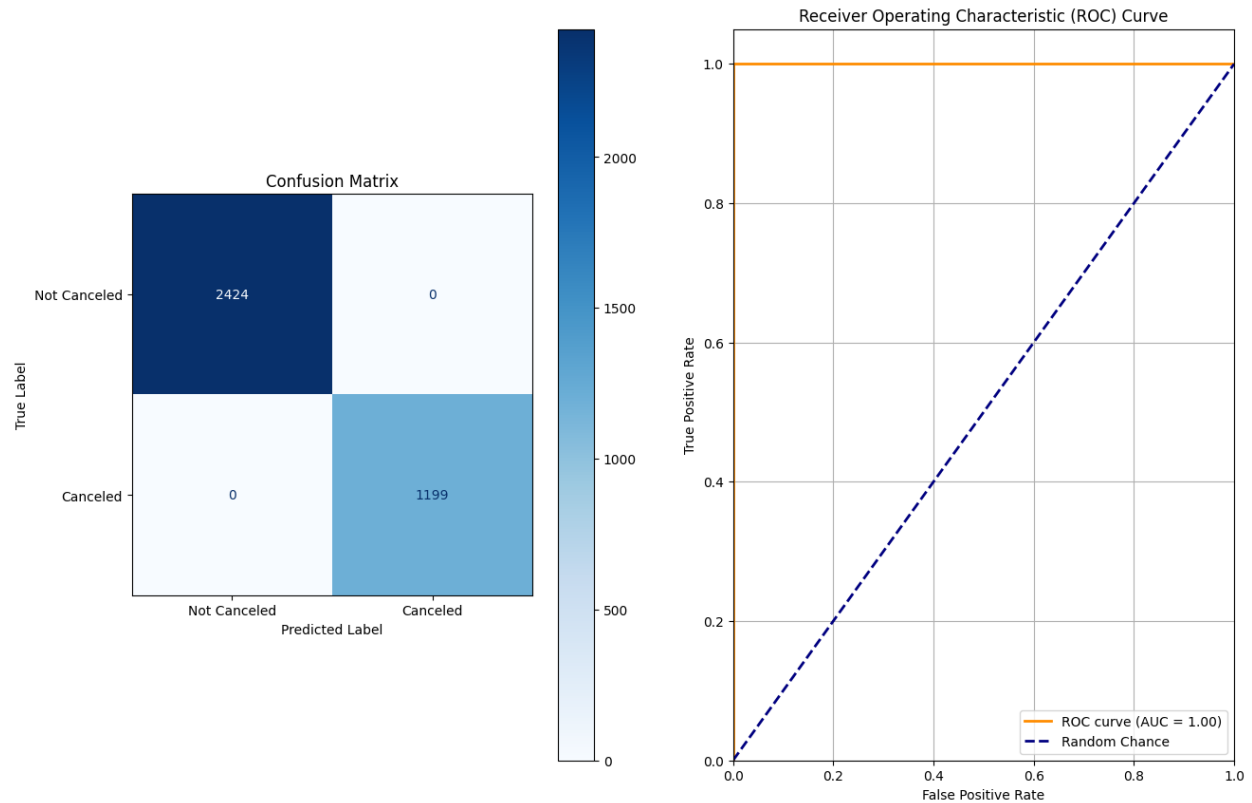
model_4 = ModelHandler(
    X = X,
    Y = Y,
    architecture = model_4,
    optimizer = optim.AdamW,
    loss_fn = nn.NLLLoss
)

model_4.train(100000, 1e-4, 100, 1e-4)
model_4.eval_model(labels)
```



Model 4 Architecture and Loss Plot

Model 4 was included specifically to test a model with only linear layers. This was to test a hypothesis that the data is highly linear, which would lead to the exceptional accuracy and training time for each of the prior models. Given that Model 4 was among the best performing models with the lowest train and inference times, it appears that the data is indeed highly linear.



Model 4 Performance Metrics

Combined Performance Metrics

Model Type	Validation Accuracy	Recall	Precision	F1-Score	Training Time (s)	Inference Time (s)
1	97.27%	91.6%	100.00%	0.96	0.0876	0.7168
2	99.97%	99.91%	100.00%	1	1.4565	1.8502
3	100.00%	100.00%	100.00%	1	0.8250	0.9488
4	100.00%	100.00%	100.00%	1	0.6358	0.8339

Performance Metrics Across Models 1, 2, 3, and 4

In the *Preliminary Analysis* phase, overfitting was identified as an issue, which drove the implementation of the early stoppage mechanic. Testing in the final stage of this analysis included an attempt to overfit the model. Typically, overfitting will occur when the training loss continues to decrease, yet the test loss plateaus. Throughout the testing, this did not appear to be the case; both the training and test losses would continue to decrease down to a slope of $1e-20$ for each of the models (except model 1, which only reached $1e-10$ within a reasonable timeframe). These models would subsequently predict on validation data with 100% accuracy, which is indicative of high linearity between the features of the data and the response variable “booking_status.”

Business Recommendations

The next stage, upon completion of model training and evaluation, is determining the use case for cancellation probability predictions output by the model. Among the possibilities, targeted retention campaigns for both high and low cancellation risk customers, as well as marketing strategies for non-customers, stand out.

The first strategy would utilize the deployed Model 3 to predict cancellation risk on all current bookings. Model 3 inherently outputs a probability distribution across “cancelled” and “not cancelled” classifications, which can be used to target the upper and lower percentiles of cancellation risk customers. The lowest cancellation risk customers could be offered promotions such as bonus points on their rewards program, referral bonuses, or access to a room upgrade. This would prove to the customer that their decision to trust ABC Hotels as a service provider was correct, strengthening the relationship. Similarly, the highest cancellation risk customers could be offered lower room rates or market beating reward opportunities to incentivize them to continue their stay with ABC Hotels.

Deployment of Model 3 is the next critical issue. The key considerations in the implementation of this model in production are noted below.

1. **Batch Deployment vs Real-Time Deployment.**

- a. Real-time deployment is preferable. This would require data pipelines to be built in order to feed new observations into the model with the variables pre-encoded. Additionally, identifying observations that may be outliers would be difficult, but the long-term maintenance section includes a solution.
- b. Batch Deployment is less efficient. The model can be run on new observations as they are populated in ABC Hotel's database versus in batches due to a simplistic data pre-processing method. Batching may reduce the feasibility of automating promotional offers.

2. **Long Term Maintenance:**

- a. Develop a monitoring dashboard to track the model's performance over time. Tools like Power BI or Tableau can be used to visualize trends in real time.
- b. Periodically launch programs targeting a small group of at-risk customers to measure the ROI of the retention campaigns.
- c. Identify interval for fine-tuning or retraining. As the hotel's customer base grows or customer behavior changes, the model must be re-fit to the new data to continue to be accurate.

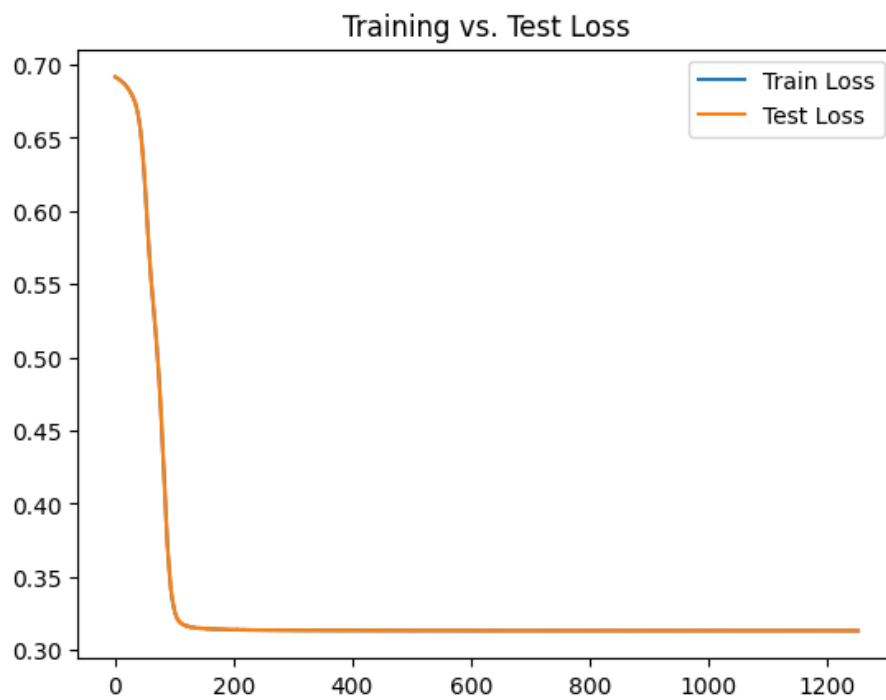
Model 3 is a powerful tool for predicting whether a customer will cancel their booking, however, the reason why must be interpreted by someone that deeply understands the data. Additionally, the data that the model was trained on is static, and future economic factors such as recession risk or competition are not represented.

An opportunity for improvement could be to include more semantic data in the data set, and subsequently the model. Questionnaires providing customer feedback on features such as pricing, customer service, or product specific reviews could provide predictive benefit and empower more pointed campaigns. Additionally, as stated in the long-term maintenance suggestions, above, periodic model tuning on new data can help mitigate prediction error due to macroeconomic factors.

Appendices



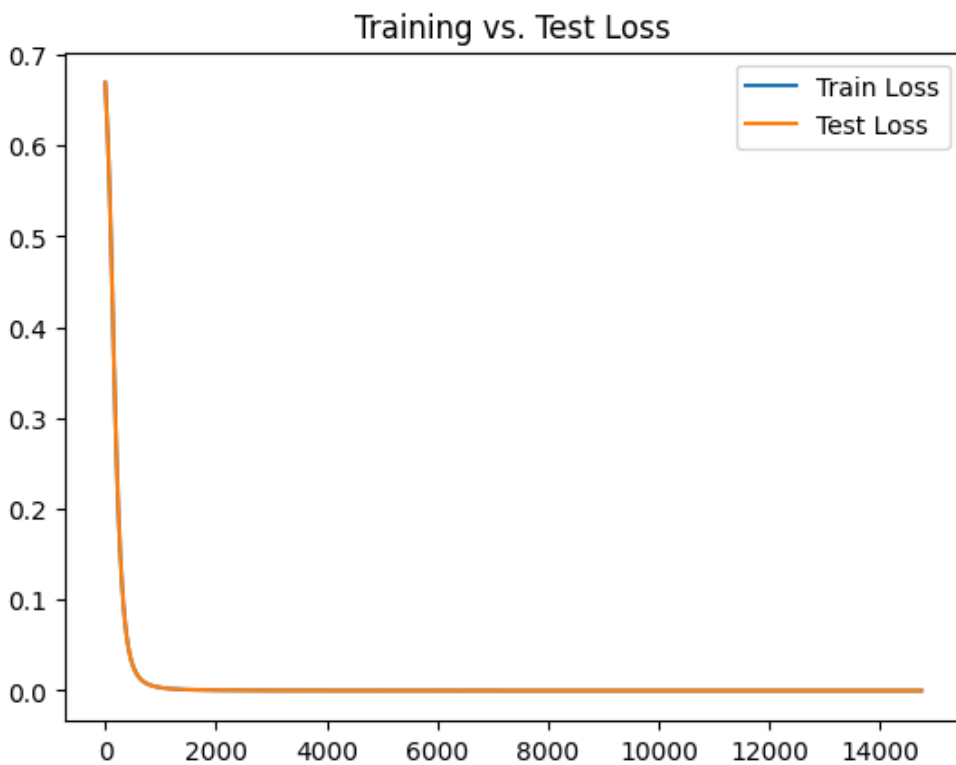
Model 1 Overfitting Test Learning Curve



Model 2 Overfitting Test Learning Curve



Model 3 Overfitting Test Learning Curve



Model 4 Overfitting Test Learning Curve